

TP 3 : Vision par ordinateur

La **vision par ordinateur (Computer Vision)** permet aux machines de **comprendre et d'interpréter des images ou des vidéos**, de la même manière que les humains le font avec leurs yeux et leur cerveau.

Étapes principales du traitement d'images

1. **Acquisition de l'image**
 - Capture via caméra, webcam ou fichier image.
2. **Prétraitement**
 - Conversion en niveaux de gris, réduction du bruit, filtrage.
3. **Segmentation**
 - Identifier les zones d'intérêt (objets, contours...)
4. **Extraction de caractéristiques**
 - Couleur, forme, texture, bords, etc.
5. **Classification / Reconnaissance**
 - Identifier ce que représente une région (chat, visage, voiture...)
6. **Interprétation**
 - Comprendre la scène globale (ex : “une personne traverse la rue”).

Objectifs

À la fin de ce TP, l'étudiant sera capable de :

1. Comprendre les concepts de base de la vision par ordinateur.
2. Manipuler des images et des vidéos avec **OpenCV**.
3. Détecter et segmenter des objets avec **Detectron2**.
4. Détecter et reconnaître des visages avec **Dlib**.

Introduction à OpenCV pour le traitement d'images et de vidéos

OpenCV (Open Source Computer Vision Library) est la bibliothèque la plus populaire pour la vision par ordinateur.

Site officiel : <https://opencv.org/>

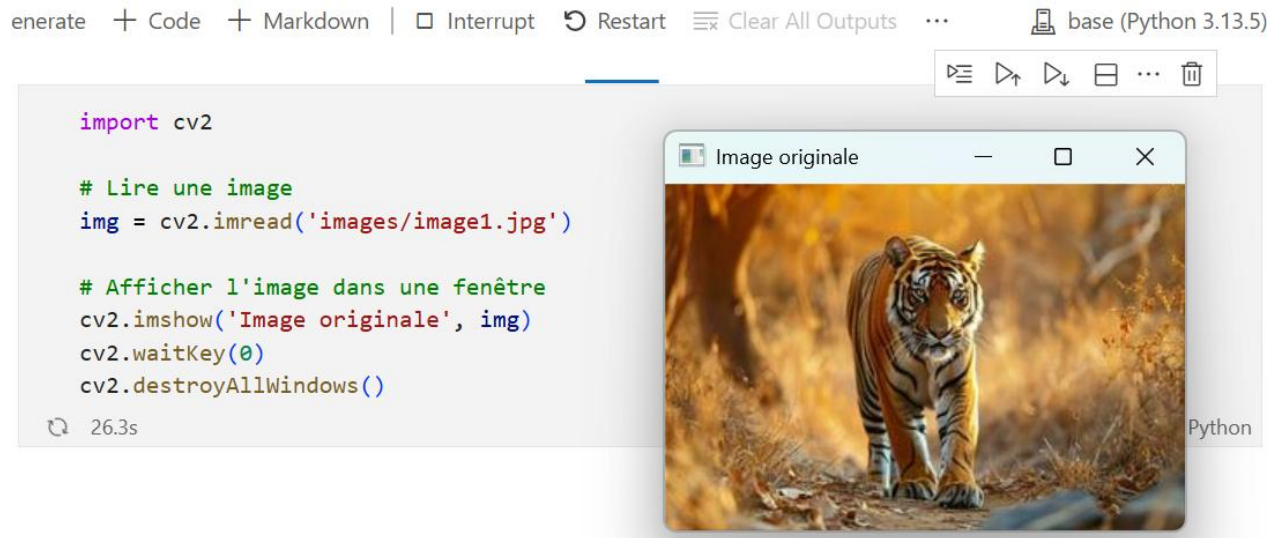
Installation: `pip install opencv-python`

Partie 1 : Concepts de base de la vision par ordinateur

Exercice 1 : Analyse d'une image

1. Choisissez une image de votre choix (ex : image.jpg).
2. Affichez-la en utilisant OpenCV.
3. Convertissez-la en niveaux de gris et affichez le résultat.
4. Appliquez un filtre de détection de contours (Canny) et affichez le résultat.

Lecture et affichage d'une image :



Conversion en niveaux de gris :



Détection des contours :



Questions :

1. Que représente la matrice retournée par OpenCV pour l'image ?
2. Pourquoi convertit-on souvent l'image en niveaux de gris avant traitement ?

Partie 2 : Traitement d'images et vidéos avec OpenCV

Exercice 2 : Lecture vidéo / webcam

1. Affichez la vidéo de votre webcam.
2. Ajoutez un filtre pour convertir la vidéo en niveaux de gris en temps réel.
3. Fermez la vidéo quand l'utilisateur appuie sur q.



Question :

- Que se passe-t-il si vous utilisez une vidéo enregistrée à la place de la webcam ?

Partie 3 : Détection et segmentation d'objets avec Detectron2

Exercice 3 : Détection d'objets

1. Chargez une image contenant plusieurs objets (`image_objects.jpg`).
2. Utilisez un modèle pré-entraîné Mask R-CNN de Detectron2 pour détecter et segmenter les objets.
3. Affichez l'image avec les masques colorés et les étiquettes.

Définition

Detectron2 est une bibliothèque développée par **Facebook AI Research (FAIR)**, spécialisée dans :

- la **détection d'objets**,
- la **segmentation d'instance** (délimiter chaque objet),
- et la **segmentation panoptique**.

Site officiel :

- <https://github.com/facebookresearch/detectron2>
- <https://detectron2.readthedocs.io/en/latest/tutorials/install.html>

Installation : `pip install 'git+https://github.com/facebookresearch/detectron2.git'`

Ou : `conda install pytorch torchvision torchaudio cpuonly -c pytorch`

<code>https://colab.research.google.com/</code>
<code># Installe Detectron2 correspondant à ta version de torch</code> <code>!pip install -U 'git+https://github.com/facebookresearch/detectron2.git'</code>
<code># Uploader une image</code> <code>from google.colab import files</code> <code>uploaded = files.upload()</code>
<code>import cv2</code> <code>import torch</code> <code>from detectron2.engine import DefaultPredictor</code> <code>from detectron2.config import get_cfg</code> <code>from detectron2 import model_zoo</code> <code>from detectron2.utils.visualizer import Visualizer</code> <code>from detectron2.data import MetadataCatalog</code> <code>from google.colab.patches import cv2_imshow</code>

```
# Configuration du modèle pré-entraîné
cfg = get_cfg()
cfg.merge_from_file(model_zoo.get_config_file(
    "COCO-InstanceSegmentation/mask_rcnn_R_50_FPN_3x.yaml"
))
cfg.MODEL.ROI_HEADS.SCORE_THRESH_TEST = 0.5
cfg.MODEL.WEIGHTS = model_zoo.get_checkpoint_url(
    "COCO-InstanceSegmentation/mask_rcnn_R_50_FPN_3x.yaml"
)

predictor = DefaultPredictor(cfg)

# Charger une image
img = cv2.imread("image1.jpg")

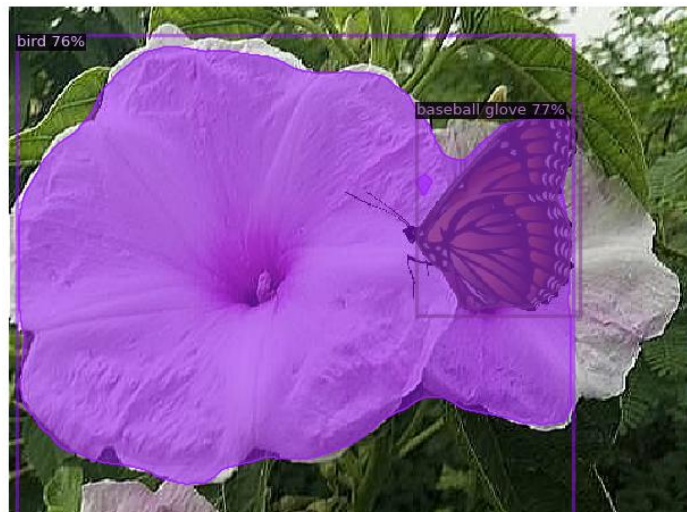
# Prédiction
outputs = predictor(img)

# Visualisation
v = Visualizer(img[:, :, ::-1],
    MetadataCatalog.get(cfg.DATASETS.TRAIN[0]), scale=1.2)
out = v.draw_instance_predictions(outputs["instances"].to("cpu"))
# cv2.imshow("Détection d'objets", out.get_image()[:, :, ::-1])
# cv2.waitKey(0)
# cv2.destroyAllWindows()
from google.colab.patches import cv2_imshow

cv2_imshow(out.get_image()[:, :, ::-1])
```

Résultat attendu : objets détectés avec leurs **masques colorés et étiquettes** (personne, voiture, chat...).

La sortie du program:





Questions :

1. Quels objets ont été détectés ?
2. Modifiez le seuil `SCORE_THRESH_TEST` et observez l'effet sur la détection.

Partie 4 : Détection et reconnaissance faciale avec Dlib

Exercice 4 : Détection faciale

1. Chargez une image contenant un ou plusieurs visages (`people.jpg`).
2. Détectez tous les visages et entourez-les de rectangles verts.

Définition

Dlib est une bibliothèque Python/C++ très performante utilisée pour :

- la **détection de visages**
- la **reconnaissance faciale**
- la **reconnaissance de points caractéristiques (landmarks)**

Installation : `pip install dlib face_recognition`

Exemples de détection de visages

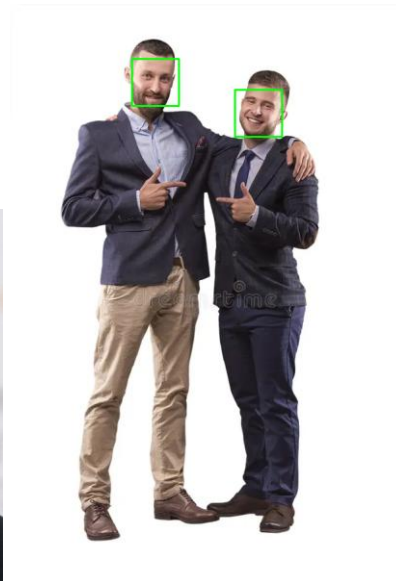
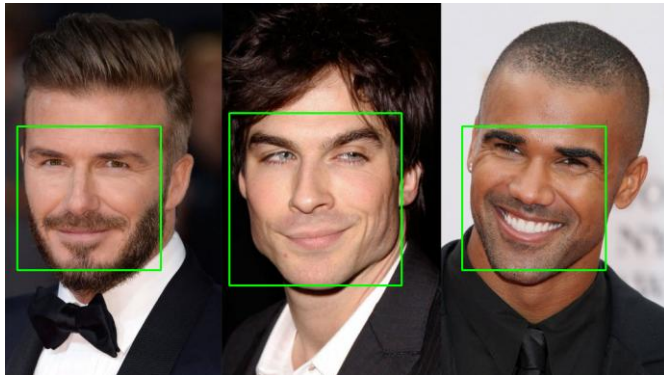
```
import dlib
import cv2

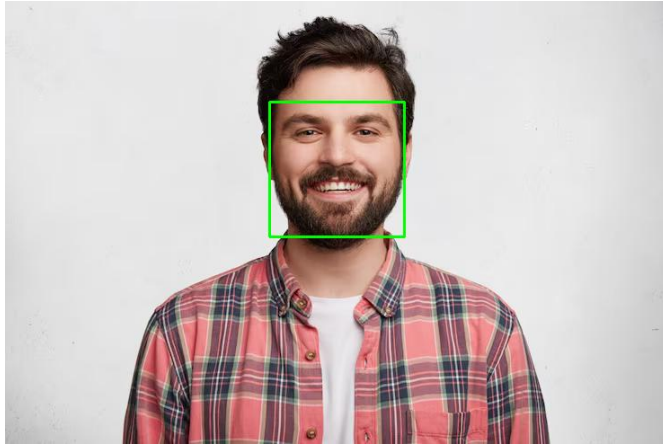
detector = dlib.get_frontal_face_detector()
img = cv2.imread("people.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

faces = detector(gray)
for face in faces:
    x, y, w, h = face.left(), face.top(), face.width(), face.height()
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)

# cv2.imshow("Visages détectés", img)
# cv2.waitKey(0)
# cv2.destroyAllWindows()
from google.colab.patches import cv2_imshow
cv2_imshow(img)
```

Output :





Exercice 5 : Reconnaissance faciale

1. Chargez une image de référence (ref.jpg) et une image test (test.jpg).
2. Comparez les visages et indiquez si c'est la même personne.

Reconnaissance faciale (avec face_recognition)

```
# Installer les dépendances
!apt-get install -y cmake libopenblas-dev liblapack-dev

# Installer dlib et face_recognition
!pip install dlib face_recognition
```

```
!pip uninstall -y dlib
!pip install dlib-bin
```

 **WARNING: Skipping dlib as it is not installed.**
Requirement already satisfied: dlib-bin in /usr/local/lib/python3.12/dist-packages (20.0.0)

```
import face_recognition
import os

# Désactiver CUDA pour dlib
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"

# Charger les images
image_ref = face_recognition.load_image_file("personne1.avif")
encodage_ref = face_recognition.face_encodings(image_ref)[0]

image_test = face_recognition.load_image_file("personne1.avif")
encodage_test = face_recognition.face_encodings(image_test)[0]

# Comparer
result = face_recognition.compare_faces([encodage_ref], encodage_test)
print("Même personne ?" , result[0])
```

Même personne ? True

```
import face_recognition
import os

# Désactiver CUDA pour dlib
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
```

```
# Charger les images
image_ref = face_recognition.load_image_file("Jim_Carrey_2008.jpg")
encodage_ref = face_recognition.face_encodings(image_ref)[0]

image_test = face_recognition.load_image_file("Jim_Carrey_2010.jpg")
encodage_test = face_recognition.face_encodings(image_test)[0]

# Comparer
result = face_recognition.compare_faces([encodage_ref], encodage_test)
print("Même personne ?" , result[0])
```

Output : Même personne ? True

Questions :

1. Que se passe-t-il si l'image test contient plusieurs visages ?
2. Quelle précision attendez-vous de ce système dans des conditions réelles (lumière, angles différents) ?

Travail à rendre

- Notebook ou script Python avec les codes complétés.
- Capture d'écran des résultats pour chaque exercice.
- Réponses aux questions posées.